Write a program to:
Calculate current need of resources by each process.
Find whether these processes can be executed with available resources. If yes,show the correct order of process execution.
To check whether the given system is in a safe state using Banker's Algorithm and to determine the safe sequence o processes.

CODE:

```
#include <stdio.h>

int main() {

 int n = 5; // Number of processes
 int m = 4; // Number of resources

 int i, j, k;

 // Allocation Matrix
 int allocation[5][4] = {
 {0,0,1,4}, // P0 - Google Drive
 {0,6,3,2}, // P1 - Firefox
 {0,0,1,2}, // P2 - Word Processor
 {1,0,0,0}, // P3 - Excel
 {1,3,5,4} // P4 - PowerPoint
 };

 // Maximum Matrix
 int max[5][4] = {
 {0,6,5,6},
 {0,6,5,2},
 {0,0,1,2},
 {1,7,5,0},
 {2,3,5,6}
 };

 // Available Resources
 int available[4] = {1,6,2,0};

 int need[5][4];
 int finish[5] = {0};
 int safeSequence[5];
 int count = 0;

// Calculate Need Matrix
 for(i = 0; i < n; i++) {
 for(j = 0; j < m; j++) {
 need[i][j] = max[i][j] - allocation[i][j];
 }
 }


 }
```

```
/ 🔹 Print Allocation Table
printf("\nAllocation Matrix:\n");
printf("Process Printer ROM HardDisk RAM\n");
for(i = 0; i < n; i++) {
printf("P%d ", i);
for(j = 0; j < m; j++) {
printf("%d ", allocation[i][j]);
}
printf("\n");

// 🔹 Print Max Table
printf("\nMax Matrix:\n");
printf("Process Printer ROM HardDisk RAM\n");
for(i = 0; i < n; i++) {
printf("P%d ", i);
for(j = 0; j < m; j++) {
printf("%d ", max[i][j]);
}
printf("\n");
}

// 🔹 Print Need Table
printf("\nNeed Matrix (Max - Allocation):\n");
printf("Process Printer ROM HardDisk RAM\n");
for(i = 0; i < n; i++) {
printf("P%d ", i);
for(j = 0; j < m; j++) {
printf("%d ", need[i][j]);
}
printf("\n");
}

// 🔹 Print Available Resources
printf("\nAvailable Resources:\n");
printf("Printer: %d\nROM: %d\nHardDisk: %d\nRAM: %d\n",
available[0], available[1], available[2], available[3]);

// 🔹 Banker's Algorithm
while(count < n) {
int found = 0;

for(i = 0; i < n; i++) {
if(finish[i] == 0) {

int flag = 0;
for(j = 0; j < m; j++) {
if(need[i][j] > available[j]) {
flag = 1;
break;
}
}
```

```
if(flag == 0) {
 for(k = 0; k < m; k++) {
 available[k] += allocation[i][k];
 }

 safeSequence[count++] = i;
 finish[i] = 1;
 found = 1;
 }
 }
 }

 if(found == 0) {
 printf("\nSystem is NOT in Safe State\n");
 return 0;
 }
 }

 printf("\nSystem is in SAFE STATE\n");
 printf("Safe Sequence: ");

 for(i = 0; i < n; i++) {
 printf("P%d ", safeSequence[i]);
 }

 printf("\n");

 return 0;
}
```

Output:

```
Allocation Matrix:
Process  Printer ROM HardDisk RAM
P0       0      0      1      4
P1       0      6      3      2
P2       0      0      1      2
P3       1      0      0      0
P4       1      3      5      4

Max Matrix:
Process  Printer ROM HardDisk RAM
P0       0      6      5      6
P1       0      6      5      2
P2       0      0      1      2
P3       1      7      5      0
P4       2      3      5      6

Need Matrix (Max - Allocation):
Process  Printer ROM HardDisk RAM
P0       0      6      4      2
P1       0      0      2      0
P2       0      0      0      0
P3       0      7      5      0
P4       1      0      0      2

Available Resources:
Printer: 1
ROM: 6
HardDisk: 2
RAM: 0

System is in SAFE STATE
Safe Sequence: P1 P2 P3 P4 P0
```